

1. Historien til SSL / TLS

For å forstå dagens TLS-protokoller, må vi først se på utviklingen fra SSL til TLS. Transport Layer Security (TLS) er en protokoll som brukes for å sikre kommunikasjon over TCP/IP, for eksempel webtrafikk via HTTPS, e-posttransport og filoverføring. Protokollen tilbyr autentisering av partene samt etablering av en kryptert sesjonsnøkkel som brukes til å beskytte videre kommunikasjon. TLS er egentlig en videreutvikling av Secure Sockets Layer (SSL), som ble utviklet av Netscape på 90-tallet. Her er en oppsummering av historien til protokollen.

Versjon	År
SSL v2	1995
SSL v3	1996
TLS 1.0	1999
TLS 1.1	2006
TLS 1.2	2008

(Boyd et al., 2020, s. 241-242)

SSL v3 ble fort ansett som usikker, og protokollen redesignet til SSL v3. Senere tok Internet Engineering Task Force (IETF) over arbeidet for standardisering og publiserte TLS 1.0 som en videreføring av SSL v3. (Boyd et al., 2020, s. 242) TLS 1.1 introduserte blant annet endringer i håndteringen av initialiseringsvektorer og padding i CBC-kryptering for å beskytte mot angrep som tidligere hadde blitt identifisert. TLS 1.2 videreutviklet protokollen ved å forbedre den kryptografiske pseudo-random-funksjonen og ved å legge til nye krypteringsmoduser som AES-GCM og AES-CCM. (Boyd et al., 2020, s. 252)

Et viktig poeng er at TLS-protokollen støtter et stort antall ciphersuites, som er kombinasjoner av nøkkelutveksling, autentisering, kryptering og hash-funksjoner. IETF har standardisert over 320 forskjellige ciphersuites. Dette gir fleksibilitet, men øker også kompleksiteten og muligheten for feilkonfigurasjoner. (Boyd et al., 2020, s. 253)

Historien til TLS viser derfor tydelig hvordan protokollen har blitt forbedret over tid, som ofte har vært et direkte svar på nye sikkerhetsangrep og kryptografiske svakheter som har blitt oppdaget.

2. Angrep mot tidligere versjoner av TLS (1.0 og 1.1)

En av de mest kjente sårbarhetene i eldre TLS-versjoner er BEAST-angrepet (Browser Exploit Against SSL/TLS).

Dette er et angrep som utnytter en svakhet i hvordan blokkchiffer i CBC-modus brukes i SSL v3 og TLS 1.0. I disse protokollene blir initialiseringsvektoren for en ny forespørsel i samme TLS-forbindelse, satt til den siste ciphertext-blokken fra forrige forespørsel. Denne mekanismen gjør det mulig for en angriper å gjennomføre et adaptivt chosen-plaintext-angrep, hvor angriperen kan teste gjetninger om verdien av bestemte plaintext-blokker. (Boyd et al., 2020, s. 268) I 2011 demonstrerte Rizzo og Duong et praktisk angrep kalt BEAST (Browser Exploit Against SSL/TLS) som kunne hente ut korte hemmelige verdier, som f.eks. cookies, ut fra HTTPS-trafikk (Boyd et al., 2020, s. 268-269). Som et svar på dette problemet, la TLS 1.1 til eksplisitte initialiseringsvektorer i CBC-modus, noe som forhindrer BEAST på en effektiv måte. (Boyd et al., 2020, s. 268-270) Et lignende angrep er POODLE, som ble publisert i 2014. Dette er et angrep som utnytter hvordan padding håndteres i CBC-kryptering i SSL v3, som da ikke er beskyttet av MAC-en. Dette gjør det mulig for en angriper å bruke en padding-oracle til å gradvis avsløre plaintext-data. (Boyd et al., 2020, s. 270)

Disse angrepene bidro til at eldre protokoller som SSL v3 og TLS 1.0 ble faset ut

3. Svakheter og angrep mot TLS 1.2

TLS 1.2 representerte en betydelig forbedring sammenlignet med tidligere versjoner, men protokollen var fremdeles utsatt for forskjellige type angrep. En viktig kategori er angrep som utnytter samspillet mellom kryptografiske algoritmer og protokollens struktur.

F.eks beskriver boka hvordan FREAK og Logjam-angrepene utnytter at autentiseringen av handshake-transkriptet i TLS frem til versjon 1.2 er basert på en MAC som beregnes fra en nøkkel, som forhandles internt i protokollen. Dette gjør det mulig for en angriper å utføre downgrade-angrep, hvor angriperen forsøker å lure brukeren og serveren til å bruke en svakere kryptografisk parameter, før autentiseringen er fullført. (Boyd et al., 2020, s. 266) (Boyd et al., 2020, s. 274)

Logjam-angrepet demonstrerte også hvordan svake Diffie-Hellmann parametere kan utnyttes. Mange servere brukte standardiserte 512-bit grupper, og med nok forhåndsberegning, kunne diskrete logaritmer i slike grupper beregnes temmelig fort. (Boyd et al., 2020, s. 266)

TLS-protokollen støttet også en svært stor mengde ciphersuites, som eldre algoritmer som RC4, DES og 3DES. Dette førte til fleksibilitet, men bidro også til et større angreps potensial, da ikke alle algoritmer var like sikre. (Boyd et al., 2020, s. 264)

I tillegg har det vært flere angrep mot spesifikke funksjoner i TLS, som f.eks renegotiation-angrepet som ble publisert i 2009. Dette angrepet utnyttet hvordan TLS tillot partene i å starte en ny handshake innenfor en allerede eksisterende forbindelse, som førte til standardisering av nye mottiltak.

Et viktig poeng her, er at mange av de mest kjente TLS-sårbarhetene ikke nødvendigvis var på grunn av én enkelt kryptografisk svakhet, men heller samspillet mellom kompleks protokolldesign, bakoverkompatibilitet, og store antall valgbare algoritmer. Dette er en av

hovedgrunnene til at TLS 1.3 senere ble designet som en mye enklere protokoll med færre mulige konfigurasjoner.

4. Kryptografien i TLS 1.3

TLS 1.3, som ble standardisert i august 2018, er en større revisjon av tidligere TLS-versjoner. Den ble utviklet for å redusere kompleksitet, fjerne usikre mekanismer og styrke den kryptografiske sikkerheten. For å forstå kryptografien i TLS 1.3, er det nyttig å ta utgangspunkt i hvordan TLS generelt er bygd opp. De to viktigste delprotokollene i TLS er handshake-protokollen og record-layer-protokollen. I handshake-protokollen blir partene enige om kryptografiske parametere, utveksler autentiseringsinformasjon, etablerer en delt hemmelighet og avleder nøkler. Deretter brukes record layer-protokollen til å transportere både handshake-meldinger og applikasjonsdata, og til å beskytte disse med autentisering og kryptering. Dette gjelder også i TLS 1.3, men protokollen er betydelig modernisert sammenlignet med TLS 1.2.(Boyd et al., 2020, s. 285-286)

En grunnleggende endring i TLS 1.3 er at protokollen ikke lenger støtter RSA-nøkkeltransport eller statisk Diffie–Hellman. Nøkkelutveksling skjer i stedet med (EC)DHE, altså ephemeral Diffie–Hellman eller elliptisk-kurve Diffie–Hellman. Dette gir “forward secrecy.” Det betyr at selv om serverens langsiktige private nøkkel skulle bli kompromittert i ettertid, kan ikke tidligere opptatt trafikk dekrypteres. Dette er en viktig forbedring fra eldre TLS-versjoner, hvor enkelte ciphersuites ikke ga slik beskyttelse.(Boyd et al., 2020, s. 286)

TLS 1.3 bruker også bare AEAD-kryptering for å beskytte data i record-protokollen. Eldre konstruksjoner med separat kryptering og MAC, som CBC og HMAC, er fjernet. I stedet brukes ciphersuites som kombinerer konfidensialitet og integritet i én og samme algoritme, for eksempel AES-GCM, AES-CCM og ChaCha20-Poly1305. Dette reduserer både kompleksiteten og risikoen for feil som tidligere kunne oppstå i samspillet mellom kryptering, padding og MAC-verifikasjon.(Boyd et al., 2020, s. 253,)

I TLS 1.3 starter nøkkelutvekslingen med at klienten sender en ClientHello-melding. Denne inneholder blant annet støttede kryptografiske grupper, ciphersuites og en key share. Serveren svarer med ServerHello og sin egen key share, slik at partene kan beregne en felles Diffie–Hellman-hemmelighet. Denne hemmeligheten brukes ikke direkte som sesjonsnøkkel, men som grunnlag for videre nøkkelutledning til egne nøkler for handshake-trafikk og applikasjonsdata.(Boyd et al., 2020, s. 244,)

Serveren autentiserer seg så normalt ved hjelp av et X.509-sertifikat og en digital signatur som binder serverens identitet til nøkkelutvekslingen. I tillegg sender partene Finished-meldinger som bekrefter at begge har samme oppfatning av handshaken og de avtalte parametere. Dette beskytter mot manipulering av handshake-forløpet og styrker sikkerheten mot blant annet downgrade-angrep.(Boyd et al., 2020, s. 246-247,)

5. Beskrivelse av HANDSHAKE og RECORD protokollene (1.3)

TLS-protokollen består av flere underprotokoller. De to viktigste for den kryptografiske funksjonaliteten er HANDSHAKE og RECORD protokollen. Handshake brukes til å etablere

en sikker forbindelse mellom klient og server, mens record-protokollen brukes til transport og beskyttelse av data etter at forbindelsen er etablert. (Boyd et al., 2020, s. 243-244)

I Handshake-protokollen blir klient og server enige om en rekke kryptografiske parametere, altså ciphersuite. Disse utveksler autentiseringsinformasjon, etablerer en delt hemmelighet, og genererer nøkler som senere brukes til kryptering og autentisering av data. Record-protokollen sørger for selve transporten av meldinger i TLS, som inkluderer både handshake-meldinger og applikasjonsdata, og kan beskytte disse, ved å bruke autentisering og kryptering. (Boyd et al., 2020, s. 243-244)

TLS inneholder også en egen alert-protokoll som brukes til å varsle om feil eller til å avslutte forbindelsen mellom partene. Disse underprotokollene ligger over TCP/IP-stakken, hvor record-protokollen fungerer som transportlag for TLS-meldingene. (Boyd et al., 2020, s. 243)

Handshake-protokollen:

For å etablere en sikker TLS-forbindelse, må klienten og serveren gjennomføre en TLS handshake. Under denne prosessen vil partene utveksle meldinger for å bli enige om kryptografiske parametere og etablere en del nøkkel. (Boyd et al., 2020, s. 244)

I Handshake prosessen, starter det med at klienten sender en ClientHello-melding til serveren. Dette er en melding som inneholder blant annet en tilfeldig verdi (nonce) og en liste over ciphersuites som klienten støtter. Serveren svarer da med en ServerHello-melding. Denne inneholder en egen tilfeldig verdi og viser til hvilken ciphersuite som er valgt. (Boyd et al., 2020, s. 244-245)

Serveren sender som regel da et sertifikat som inneholder serverens offentlige nøkkel, slik at klienten kan autentisere serverens identitet. Og hvis ciphersuiten krever det, kan serveren også sende en ServerKeyExchange-melding, som kan inneholde en ephemeral Diffie-Hellmann-verdi, for eksempel. Serveren kan også da be om klientautentisering ved å sende en CertificateRequest-melding. (Boyd et al., 2020, s. 244)

Klienten verifiserer serverens sertifikat og sender meldinger tilbake til serveren. Dette kan inkludere et klientsertifikat, en ClientKeyExchange-melding, og eventuelt en CertificateVerify-melding. Disse meldingene brukes til å etablere en delt hemmelighet, som kalles premaster secret, som vil senere brukes til å generere de symmetriske sesjonsnøklerne. (Boyd et al., 2020, s. 244)

Når nøklene er generert, sender klienten en ChangeCipherSpec-melding, som signaliserer at alle videre meldinger vil bli beskyttet av record-protokollen. Deretter sender klienten en Finished-melding, som fungerer som en bekreftelse på at handshake-prosessen er riktig gjennomført. Serveren utfører lignende steg, og sender til slutt sin egen Finished-melding. Når alt dette er gjort, kan applikasjonsdataene sendes over den etablerte sikre forbindelsen. (Boyd et al., 2020, s. 244-245)

Record-protokollen:

Når Handshake-prosessen er ferdig, brukes TLS record-protokollen til å transportere alle meldingene i forbindelsen, samt applikasjonsdata. Record-protokollen sørger for at dataene

deles opp i mindre blokker, og at de kan bli komprimert, autentisert og kryptert før de sendes videre over nettverket. (Boyd et al., 2020, s. 249) En viktig funksjon i RECORD-protokollen, er å sikre konfidensialitet og integritet for dataene som sendes mellom klient og server. Dette oppnås ved å bruke symmetriske nøkler, som ble generert under handshake-prosessen. Dermed kan applikasjonsdata sendes på en sikker måte over en usikker nettverksforbindelse. (Boyd et al., 2020, s. 249-250)

Record-protokollen fungerer også som transportlag for andre TLS-meldinger, inkludert handshake og alert-meldinger. Den fungerer derfor som et slags fundament i TLS-arkitekturen, hvor alle andre TLS-protokoller bruker record-laget for en sikker kommunikasjon. (Boyd et al., 2020, s. 243)

6. Evt. svakheter og angrep mot TLS 1.3

Selv om TLS 1.3 er en betydelig forbedring sammenlignet med tidligere versjoner, betyr ikke det at protokollen er helt uten svakheter. Det viktigste poenget er likevel at de fleste kjente problemene med TLS 1.3 ikke skyldes at hovedkryptografien er brutt, men heller at enkelte funksjoner gir praktiske kompromisser, eller at sikkerheten fortsatt avhenger av korrekt implementasjon og konfigurasjon.

En av de viktigste svakhetene i TLS 1.3 er knyttet til 0-RTT early data. Denne funksjonen gjør det mulig for en klient å sende data veldig tidlig i forbindelsen, før hele handshaken er fullført. Fordelen er lavere forsinkelse, men ulempen er at slike data ikke har full beskyttelse mot replay-angrep. RFC 8446 understreker at TLS 1.3 ikke garanterer beskyttelse mot replay mellom forbindelser for 0-RTT-data. Det betyr at en angriper i noen tilfeller kan fange opp og sende samme tidlige forespørsel på nytt. Dette er spesielt problematisk dersom slike forespørsler fører til handlinger som innlogging, kjøp eller andre tilstandsendringer på serversiden. (Fischlin & Günther, 2017)

Samtidig er det viktig å få fram at TLS 1.3 faktisk har fjernet mange av angrepsflatene fra eldre versjoner. Renegotiation er fjernet i TLS 1.3, og dermed forsvinner også en kjent klasse med angrep som tidligere rammet TLS 1.2 og eldre. På samme måte er gamle mekanismer som RSA key transport og CBC-baserte ciphersuites tatt bort. Dette gjør at mange klassiske angrep, som BEAST, POODLE og Bleichenbacher-relaterte problemer i nøkkeltransporten, ikke lenger treffer TLS 1.3 på samme måte. (Jain, 2025)

En annen viktig svakhet ligger ikke nødvendigvis i selve protokollen, men i implementasjoner. Historisk har mange alvorlige TLS-sårbarheter kommet fra feil i biblioteker og programvare, ikke fra brudd i protokollens design. Det samme gjelder for TLS 1.3: dersom sertifikatvalidering gjøres feil, nøkler håndteres uforsvarlig, eller tilstandsmaskinen i handshaken implementeres feil, kan systemet fortsatt bli sårbart selv om protokollen i seg selv er sterk. TLS 1.3 reduserer kompleksiteten betydelig, men den fjerner ikke behovet for trygg implementasjon. (Jain, 2025)

Samlet sett kan vi derfor si at TLS 1.3 er langt sikrere enn tidligere TLS-versjoner, men ikke helt uten utfordringer. Den viktigste praktiske svakheten er replay-risiko ved 0-RTT early data. I tillegg finnes det fortsatt begrensninger knyttet til synlige metadata, bakoverkompatibilitet med eldre TLS-versjoner og mulige implementasjonsfeil.

Referanser:

Boyd, C., Mathuria, A., Stebila, D. (2020). Information Security and Cryptography: Protocols for Authentication and Key Establishment (utg. 2). Springer.

Fischlin, M., & Günther, F. (2017, April 1). Replay Attacks on Zero Round-Trip Time: The Case of the TLS 1.3 Handshake Candidates. IEEE Xplore.
<https://doi.org/10.1109/EuroSP.2017.18>

Jain, T. (2025, January 23). Understanding TLS 1.2 and TLS 1.3 | Encryption Consulting. Encryption Consulting. <https://www.encryptionconsulting.com/tls-1-2-and-tls-1-3/>